

Juniorprogrammierer.de

Go 0: Getting started

2024/25/26 – Sascha Stojanovic

Some Go facts

- Biggest misconception => its Go not Golang, Golang is based on a earlier domain name “Golang.org”
- It was developed by google 2007 and open sourced 2009
- Goal was
 - Simple and easy read language like python
 - Efficient and Safe like C++
- Why a new language
 - To create create application which can easily do thinks in parallel
 - Example youtube => watch a video, write a comment and like comments all in parallel possible

Install Go

- You need vscode
- You need Go compiler => making go readable to your pc

- Before compiler

```
[\\W]$ go  
zsh: command not found: go
```

- <https://go.dev/doc/install>
 - After installation of compiler

```
● D067975 /Users/D067975/go 16:36 go version  
go version go1.25.7 darwin/arm64
```

- In settings of vscode execute this cmd “Go: Install/Update Tools”

Your first hello world

- In your terminal enter “go mod init example.com/hello” you will learn about it later on
- Create a hello.go-file and enter this code

```
package main

import ("fmt")

func main() {
    fmt.Println("Hello World!")
}
```

- Now run your code

Go Syntax and Statement

- A Go file consist of 4 elements (based on example hello.go)
 - Package declaration => Every program is part of a package here it is main
 - Import packages => `fmt` import let us use the `fmt` functionalities
 - Functions => `func main() {}` is a function and code inside `{}` is executed
 - Statement/Expression => Stuff inside a function
- Any executable code in go belongs to main package
- `fmt.Println("Hello World!")` is a statement
 - statements are separated by ending a line (hitting the Enter key) or by a semicolon `;`
 - The left curly bracket `{` cannot come at the start of a line

Comments, datatypes, variable, const

- Like in java they start with “//”
- Like in java lets focus to the most important ones
 - int, float32, string, bool
- To define a variable, follow this pattern
 - “var + name + datatype = value”
 - Example: var student1 string = ”John”
 - Like in python Go can autoassign a type based on the value
 - Example : var student2 = “Jane” => type is inferred from value
 - Using “:=”-sign pattern you even don’t need to use “var” anymore
 - Example: student3 = “Josh”
- For unchangeable/read-only cases use “const”
 - Example: “const PI = 3.14”

Arrays

- Array with a defined length
 - `var arr1 = [3]int{1,2,3}`
- Array with an inferred length
 - `arr2 = [...]int{1,2,3}`
- Getting a value of an array or overwriting it on a position
 - `arr2[1] = 20`
- The length of an array is the same as in python
 - `len(arr1)`
- What is the problem of an array? It is ...

Slices

- Our dynamic growing or shrinking array
 - Empty one `myslice1 := []int{}`
 - This one is prefilled `myslice2 := []string{"Go", "Slices", "Are", "Powerful"}`
- Overwriting or assigning a value to a certain index works as in arrays
- With append you can dynamically add values to an array
 - `myslice1 = append(myslice1, 20, 21)`
- To remove you can use the delete function
 - `slices.Delete(s, startIdx, endIdx)`

Conditions

- Conditions are the same as in Java but without “()”

```
if x < 0 {  
    fmt.Println("x ist negativ")  
} else if x == 0 {  
    fmt.Println("x ist null")  
} else {  
    fmt.Println("x ist positiv")  
}
```

Loops

- For int loop

```
for i:=0; i < 5; i++ {  
  
    fmt.Println(i)  
  
}
```

- For each loop

```
fruits := [3]string{"apple", "orange", "banana"}  
for idx, val := range fruits {  
  
    fmt.Printf("%v\t%v\n", idx, val)  
  
}
```

Exersize

1. Create a slice with your collected treasure values (numbers between 1-100)

```
treasures := []int{15, 42, 7, 88, 23, 42, 91, 5, 67, 42}
```

2. Count and categorize the treasures (with loops and conditionals):

- How many "small treasures" (1-30)?
- How many "medium treasures" (31-70)?
- How many "large treasures" (71-100)?
- How often does the number 42 (the magic number) appear?

3. Find the most valuable treasure (with loops):

- Which number is the largest?
- At which position in the slice is it?

4. Double all small treasures (under 30):

- Loop through the slice and multiply these values by 2

5. Remove all magic 42s:

- Create a new slice without the 42s

Expected output:

 Your treasures: [15 42 7 88 23 42 91 5 67 42]

 Statistics:

Small treasures (1-30): 4

Medium treasures (31-70): 2

Large treasures (71-100): 2

Magic 42s found: 3

 Most valuable treasure: 91 at position 6

 Doubling all small treasures...

After doubling: [30 42 14 88 46 42 91 10 67 42]

 Without the magic 42s: [30 14 88 46 91 10 67]